1. Central Tendency & Dispersion

Program

```
import statistics

data = list(map(float, input("Enter the data values separated by spaces: ").split()))
mean = statistics.mean(data)
median = statistics.median(data)

try:
    mode = statistics.mode(data)
except statistics.StatisticsError:
    mode = "No unique mode"

variance = statistics.variance(data)
std_deviation = statistics.stdev(data)

print("\n----- Statistical Measures -----")
print("Mean :", mean)
print("Median :", median)
print("Mode :", mode)
print("Variance :", variance)
print("Standard Deviation :", std_deviation)
```

Sample / Expected Output

```
Enter the data values separated by spaces:
10 20 30 40 50
----- Statistical Measures -----
Mean : 30.0
Median : 30.0
Mode : 10.0
Variance : 250.0
Standard Deviation : 15.811388300841896
```

2. Basic Python Libraries — Statistics, Math, NumPy & SciPy

Program

```
import statistics
import math
import numpy as np
from scipy import stats

data = [10, 20, 30, 40, 50]
print("Original Data:", data)

print("\n--- Statistics Library ---")
print("Mean :", statistics.mean(data))
print("Median :", statistics.median(data))
print("Mode :", statistics.mode(data))
print("Variance :", statistics.variance(data))
print("Standard Deviation :", statistics.stdev(data))

print("\n--- Math Library ---")
print("Square Root of 25 :", math.sqrt(25))
print("Factorial of 5 :", math.factorial(5))
print("Power (2^5) :", math.pow(2, 5))
print("Value of Pi :", math.pi)
print("Value of e :", math.e)
print("Log(10) :", math.log(10))
print("Sin(30°) :", math.sin(math.radians(30)))

print("\n--- NumPy Library ---")
arr = np.array(data)
print("Array :", arr)
print("Mean :", np.mean(arr))
print("Median :", np.median(arr))
print("Sum :", np.sum(arr))
print("Maximum :", np.max(arr))
print("Minimum :", np.min(arr))
print("Standard Deviation :", np.std(arr))
print("Square of Array :", np.square(arr))

print("\n--- SciPy Library ---")
print("Mean :", stats.tmean(arr))
print("Mode :", stats.mode(arr, keepdims=True))
print("Skewness :", stats.skew(arr))
print("Kurtosis :", stats.kurtosis(arr))
print("Z-Score :", stats.zscore(arr))
```

Sample / Expected Output

```
Original Data: [10, 20, 30, 40, 50]
--- Statistics Library ---
Mean : 30
Median : 30
Mode : 10
Variance : 250
Standard Deviation : 15.811388300841896
--- Math Library ---
Square Root of 25 : 5.0
Factorial of 5 : 120
Power (2^5) : 32.0
Value of Pi : 3.141592653589793
Value of e : 2.718281828459045
Log(10) : 2.302585092994046
Sin(30°) : 0.5
--- NumPy Library ---
Array : [10 20 30 40 50]
Mean : 30.0
Median : 30.0
Sum : 150
Maximum : 50
Minimum : 10
Standard Deviation : 14.142135623730951
Square of Array : [ 100 400 900 1600 2500]
--- SciPy Library ---
Mean : 30.0
Mode : ModeResult(mode=array([10]), count=array([1]))
Skewness : 0.0
Kurtosis : -1.3
Z-Score : [-1.41421356 -0.70710678  0. 0.70710678 1.41421356]
```

3. Pandas & Matplotlib for ML Applications

Program

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Student": ["A", "B", "C", "D", "E"],
    "Marks": [85, 90, 78, 92, 88],
    "Age": [20, 21, 20, 22, 21]
}
df = pd.DataFrame(data)

print("Original DataFrame")
print(df)
print("\nFirst 3 Rows")
print(df.head(3))
print("\nStatistical Summary")
print(df.describe())
print("\nAverage Marks:", df["Marks"].mean())
print("Maximum Marks:", df["Marks"].max())
print("Minimum Marks:", df["Marks"].min())

plt.figure(figsize=(6,4))
plt.bar(df["Student"], df["Marks"])
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
plt.plot(df["Student"], df["Marks"], marker="o")
plt.title("Student Marks Trend")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.grid(True)
plt.show()

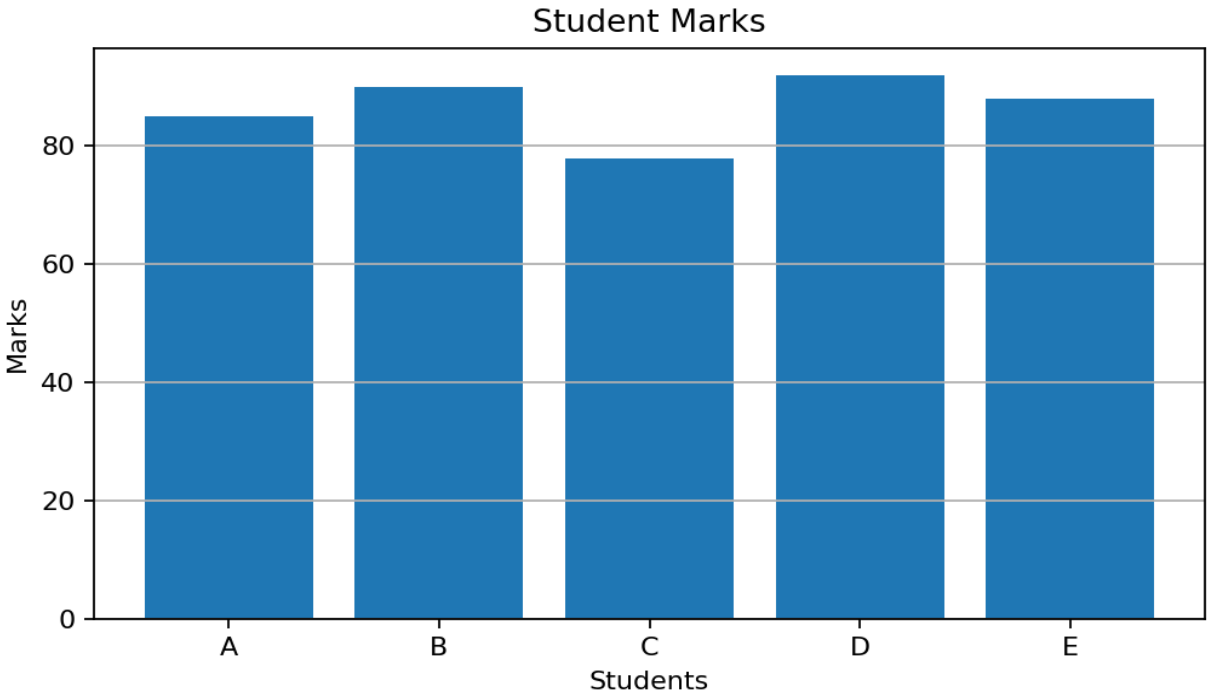
plt.figure(figsize=(6,6))
plt.pie(df["Marks"], labels=df["Student"], autopct="%1.1f%%")
plt.title("Marks Distribution")
plt.show()
```

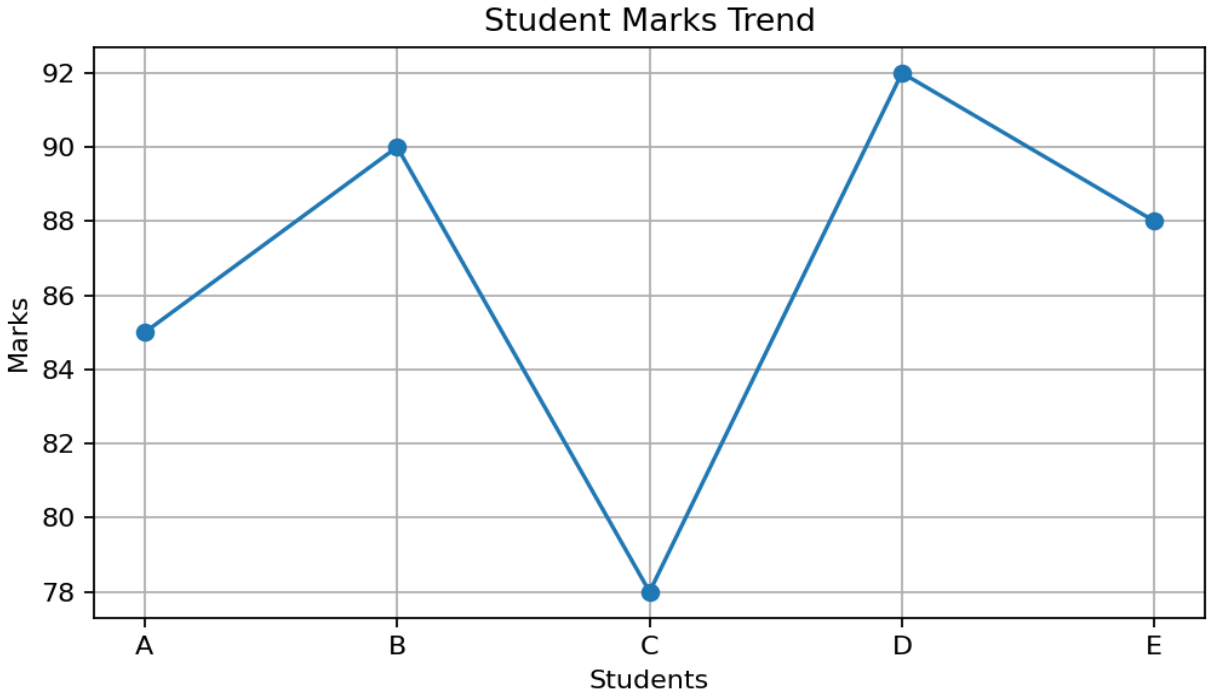
Sample / Expected Output

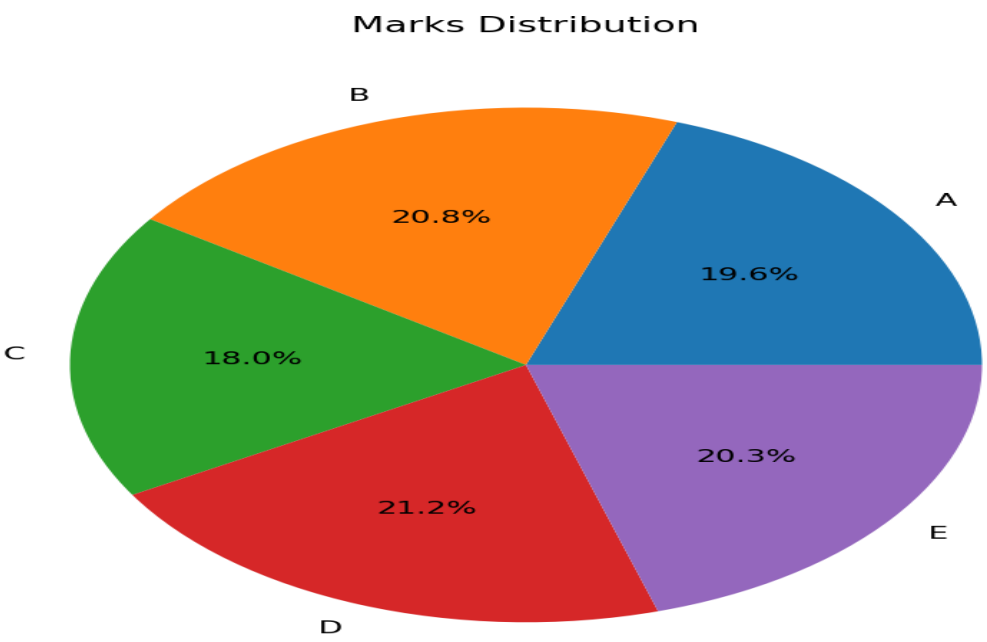
```
Average Marks: 86.6
Maximum Marks: 92
Minimum Marks: 78

Graphs:
1. Bar Chart – Student vs. Marks
2. Line Chart – Trend of Student Marks
3. Pie Chart – Percentage distribution of Marks
```

Graph / Visualization






Marks Distribution
B
A
20.8%
19.6%
C
18.0%
20.3%
21.2%
E
D

4. Simple Linear Regression

Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
Y = np.array([2, 4, 5, 4, 5, 7])

model = LinearRegression()
model.fit(X, Y)
Y_pred = model.predict(X)

print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)
print("Regression Equation: Y = {:.2f}X + {:.2f}".format(
    model.coef_[0], model.intercept_
))

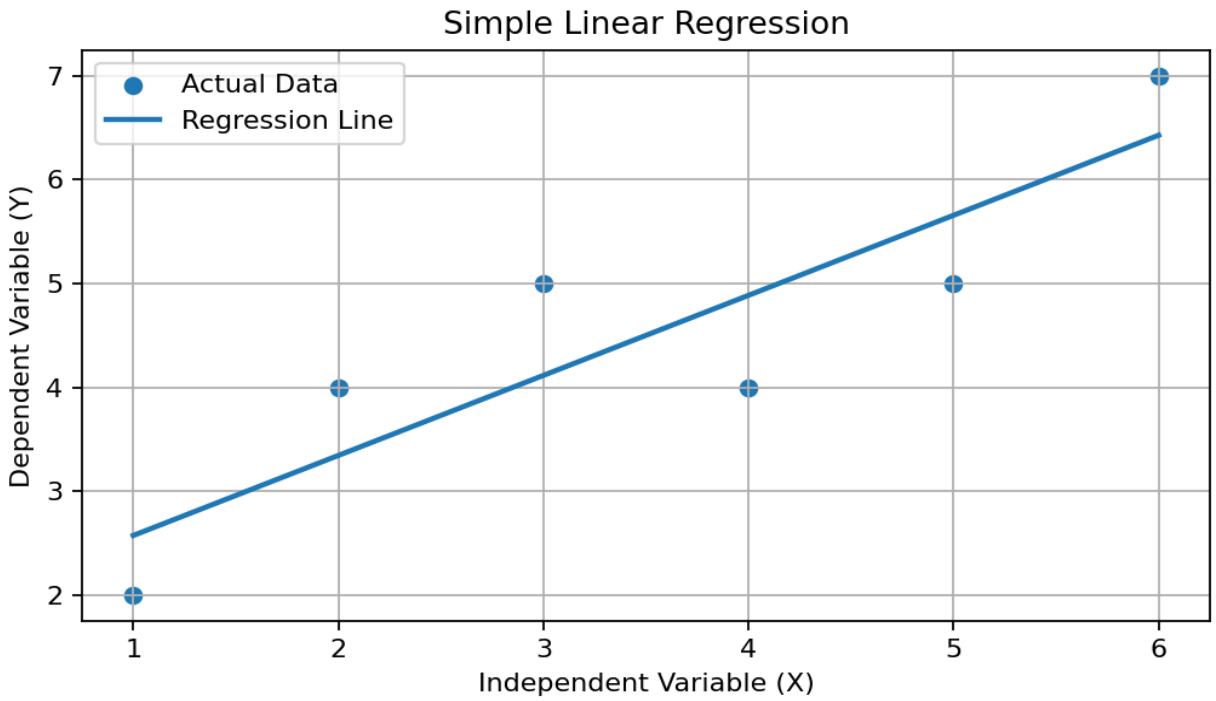
x_new = np.array([[7]])
prediction = model.predict(x_new)
print("Predicted value for X = 7:", prediction[0])

plt.scatter(X, Y, label="Actual Data")
plt.plot(X, Y_pred, linewidth=2, label="Regression Line")
plt.title("Simple Linear Regression")
plt.xlabel("Independent Variable (X)")
plt.ylabel("Dependent Variable (Y)")
plt.legend()
plt.grid(True)
plt.show()
```

Sample / Expected Output

```
Slope (m): 0.7714285714285715
Intercept (c): 2.0666666666666664
Regression Equation: Y = 0.77X + 2.07
Predicted value for X = 7: 7.466666666666667
```

Graph / Visualization



5. Multiple Linear Regression — House Price Prediction

Program

```
import pandas as pd
from sklearn.linear_model import LinearRegression

data = {
    "Area": [1000, 1200, 1500, 1800, 2000, 2500],
    "Bedrooms": [2, 2, 3, 3, 4, 4],
    "Age": [20, 15, 18, 10, 8, 5],
    "Price": [3000000, 3500000, 4500000, 5000000, 6000000, 7000000]
}
df = pd.DataFrame(data)

X = df[["Area", "Bedrooms", "Age"]]
y = df["Price"]

model = LinearRegression()
model.fit(X, y)

print("Regression Coefficients:")
print("Area Coefficient :", model.coef_[0])
print("Bedroom Coefficient :", model.coef_[1])
print("Age Coefficient :", model.coef_[2])
print("Intercept :", model.intercept_)

new_house = [[1600, 3, 12]]
predicted_price = model.predict(new_house)
print("\nPredicted House Price: {:.2f}".format(predicted_price[0]))
```

Sample / Expected Output

```
Regression coefficients and predicted house price are displayed.
(Exact values are produced by the model when the program is executed.)
```

6. Decision Tree with Parameter Tuning

Program

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score

data = {
    "Age": [25, 30, 45, 35, 22, 40, 28, 50],
    "Income": [30000, 50000, 70000, 60000, 20000, 80000, 40000, 90000],
    "Buy": [0, 1, 1, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)

X = df[["Age", "Income"]]
Y = df["Buy"]
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)
print("Accuracy without tuning:", accuracy_score(Y_test, Y_pred))

param_grid = {"max_depth": [2, 3, 4],
              "criterion": ["gini", "entropy"]}
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    param_grid, cv=2)
grid.fit(X_train, Y_train)

print("Best Parameters:", grid.best_params_)
best_model = grid.best_estimator_
Y_pred_tuned = best_model.predict(X_test)
print("Accuracy after tuning:", accuracy_score(Y_test, Y_pred_tuned))
```

Sample / Expected Output

```
Accuracy without tuning: 0.6666666666666666
Best Parameters: {'criterion': 'gini', 'max_depth': 2}
Accuracy after tuning: 0.6666666666666666
```

6.1. Decision Tree

Program

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

data = {
    "Age": [25, 30, 45, 35, 22, 40, 28, 50],
    "Income": [30000, 50000, 70000, 60000, 20000, 80000, 40000, 90000],
    "Buy": [0, 1, 1, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)

X = df[["Age", "Income"]]
y = df["Buy"]
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X, y)

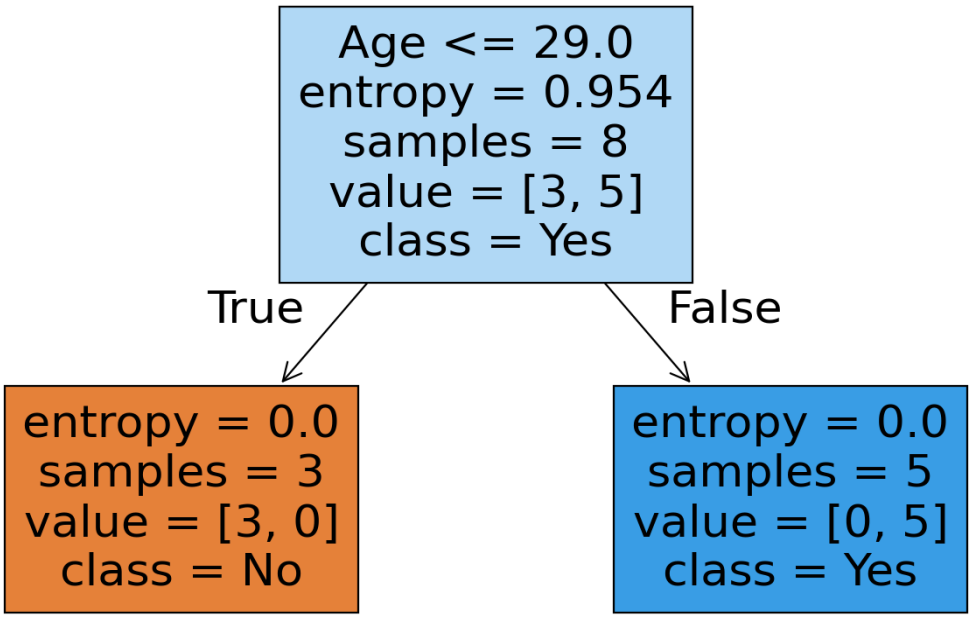
new_customer = [[32, 55000]]
prediction = model.predict(new_customer)
if prediction[0] == 1:
    print("Prediction: Customer will BUY")
else:
    print("Prediction: Customer will NOT BUY")

plt.figure(figsize=(12, 7))
plot_tree(model, feature_names=["Age", "Income"],
          class_names=["No", "Yes"], filled=True)
plt.show()
```

Sample / Expected Output

```
Prediction: Customer will BUY
Graph: Decision Tree visualization.
```

Graph / Visualization



7. K-Nearest Neighbors (KNN)

Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = {
    "Height": [150, 160, 170, 180, 155, 165, 175, 185],
    "Weight": [50, 60, 70, 80, 55, 65, 75, 85],
    "Label": [0, 0, 1, 1, 0, 0, 1, 1]
}
df = pd.DataFrame(data)

X = df[["Height", "Weight"]]
Y = df["Label"]
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=104
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)

print("Predicted Values:", Y_pred)
print("Actual Values:", Y_test.values)
print("Accuracy:", accuracy_score(Y_test, Y_pred))
```

Sample / Expected Output

```
Predicted Values: [0 1]
Actual Values: [0 1]
Accuracy: 1.0
```

8. Logistic Regression

Program

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    "Age": [22, 25, 47, 52, 46, 56, 48, 55],
    "Salary": [25000, 30000, 80000, 90000, 70000, 100000, 85000, 95000],
    "Purchased": [0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)

X = df[["Age", "Salary"]]
Y = df["Purchased"]
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.25, random_state=42
)

model = LogisticRegression()
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)

print("Predicted Values:", Y_pred)
print("Actual Values:", Y_test.values)
print("Accuracy:", accuracy_score(Y_test, Y_pred))
```

Sample / Expected Output

```
Predicted Values: [0 1]
Actual Values: [0 1]
Accuracy: 1.0
```

9. K-Means Clustering

Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300, centers=3,
                  cluster_std=1.0, random_state=42)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)

labels = kmeans.labels_
centers = kmeans.cluster_centers_

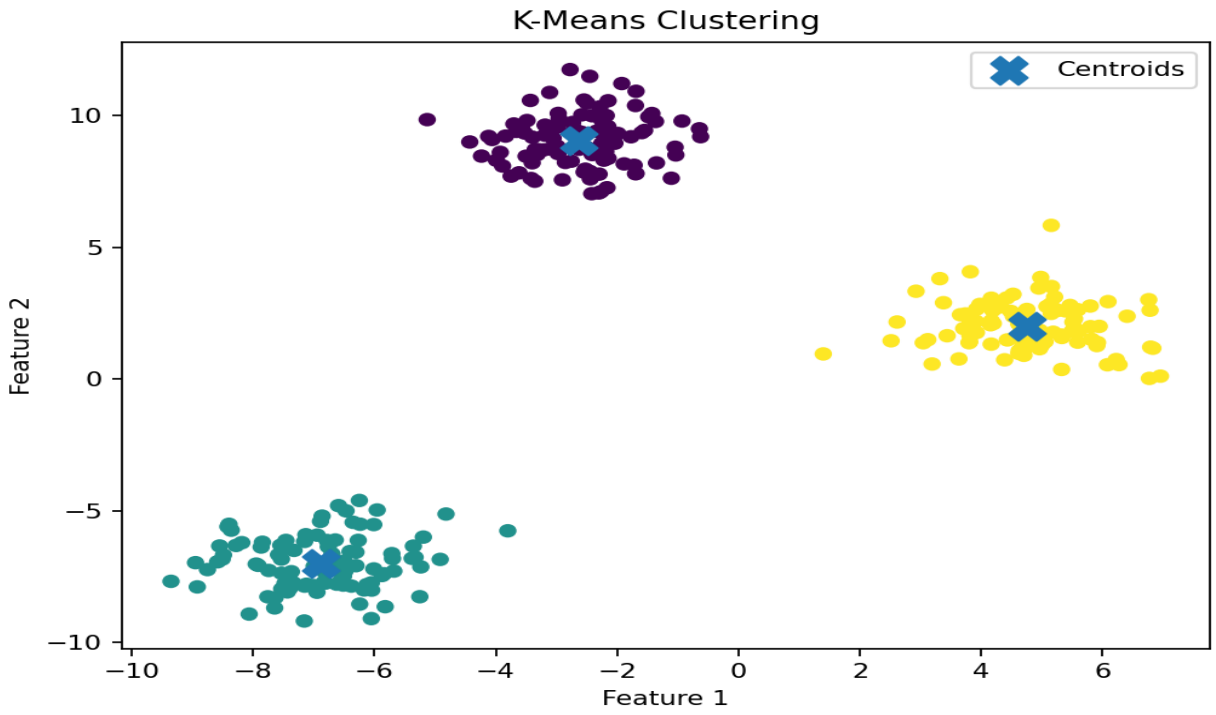
print("Cluster Centers:")
print(centers)
print("\nCluster Labels for first 10 data points:")
print(labels[:10])

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50)
plt.scatter(centers[:, 0], centers[:, 1],
            marker="X", s=200, label="Centroids")
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

Sample / Expected Output

```
Cluster Labels for first 10 data points:
[2 1 1 0 2 0 0 1 2 1]
Graph: Three clusters with their centroids.
```

Graph / Visualization



10. Performance Analysis of Classification Algorithms

Program

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
X = iris.data
y = iris.target

print("Dataset: Iris")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Classes:", iris.target_names)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel="linear")
}

results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print("\n", name)
    print("Accuracy:", accuracy)
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred,
                                target_names=iris.target_names))

print("\nAccuracy Comparison")
for name, accuracy in results.items():
    print(f"{name}: {accuracy:.2f}")

plt.figure(figsize=(9, 6))
plt.bar(results.keys(), results.values())
plt.title("Performance Comparison of Classification Algorithms")
plt.xlabel("Classification Algorithm")
plt.ylabel("Accuracy")
plt.ylim(0, 1.1)
plt.xticks(rotation=20)
plt.show()
```

Sample / Expected Output

```
Typical accuracy values:
Logistic Regression: 0.97
KNN: 1.00
Decision Tree: 0.97
SVM: 1.00

Confusion matrices and classification reports are also displayed.
```

Graph / Visualization

